CSC490 Assignment A2 - Data processing Pipelines and Infrastructure as code

By Maria Ma:1009054924, Zhixiao Fu:1009834342, Siyi Zhu:1008793076, Chloe Yang:1009261433

1 Part One: Aspirational Datasets

If we had a magic wand, the following list of datasets and data schemas is what our team would need to be successful in this project:

1. Isolated Word-Level ASL Sign Videos Training Dataset

We need many short video clips of individual ASL signs performed by diverse signers such that our video classifier can learn the spatial and temporal patterns that distinguish one sign from another.

Ideally:

- The glosses not only cover the target vocabulary of our project but also have a large scale of more than 2000 words.
- There are more than 100 videos per gloss in order to capture signer variation.
- There are more than 50 unique signers having balanced ages, genders, hand sizes, etc.
- Each video clip is tightly trimmed to the sign only, with no useless idle time.
- Each video clip has high resolution and reasonable frame rate, yet is not bulky in file size.
- The video clips contain a mix of clean and real-world backgrounds (home, office, school, restaurant, clinic, etc.)
- The video clips use a variety of lighting, camera angles, and camera types.

An ideal schema would be:

Field	Type	Description	Example
video_id	string	Unique identifier for video clip	“wlasl_hello_001”
video_path	string	Path to video clip file	“videos/hello/wlasl_hello_001 .mp4”
gloss	string	English label for the ASL sign	“hello”
category	string	Scenario category	“greeting”
signer_id	string	Unique identifier for the signer	“signer_009”

signer_demographics	object	Characteristics of the signer	{“age_range”: “20-30”, “skin_tone”: “medium”, “used_hand”: “right”, “gender”: “female”}
duration	float	Video clip duration (in seconds)	1.9
fps	int	Video clip frame rate	30
num_frames	int	Total number of frames in video clip	42
resolution	string	Video clip resolution	“1280x720”
env_type	string	Type of environment	“clean studio”
light_type	string	Type of lighting	“daylight”
camera_type	string	Type of camera	“phone front”

2. Mobile-Captured Evaluation Dataset

We need many short video clips of individual ASL signs recorded on mobile phone cameras in the way our app will actually be used (front-facing iPhone camera, held at arm's length, in real environments like a restaurant or school) such that we can test whether our model accuracy holds up under real deployment conditions.

Ideally:

- The glosses not only cover the target vocabulary of our project but also have a large scale of more than 2000 words.
- There are more than 15 videos per gloss in order to capture signer variation.
- There are more than 10 unique signers (which should not be overlapping with the ones in the training dataset) having balanced ages, genders, hand sizes, etc.
- Each video clip is captured using a front-facing mobile phone camera (selfie mode).
- Each video clip is tightly trimmed to the sign only, with no useless idle time.
- Each video clip has high resolution and reasonable frame rate, yet is not bulky in file size.
- The video clips contain various real-world backgrounds (home, office, school, restaurant, clinic, etc.)
- The video clips use a variety of lighting and camera angles.

An ideal schema would be:

Field	Type	Description	Example
video_id	string	Unique identifier for video clip	“wlasl_hello_001”
video_path	string	Path to video clip file	“videos/hello/wlasl_hello_001

			.mp4”
gloss	string	English label for the ASL sign	“hello”
category	string	Scenario category	“greeting”
signer_id	string	Unique identifier for the signer	“signer_001”
signer_demographics	object	Characteristics of the signer	{“age_range”: “20-30”, “skin_tone”: “light”, “used_hand”: “left”, “gender”: “male”}
duration	float	Video clip duration (in seconds)	1.1
fps	int	Video clip frame rate	30
num_frames	int	Total number of frames in video clip	38
resolution	string	Video clip resolution	“1280x720”
env_type	string	Type of environment	“restaurant”
light_type	string	Type of lighting	“daylight”
camera_type	string	Type of camera	“phone front”

3. Scenario-Specific Phrase Sequence Evaluation Dataset

We need many video clips of multi-sign phrases used in our everyday life—including our project’s target scenarios (e.g., "water please", "eat more")—such that we can evaluate whether our app can support realistic turn-by-turn interactions.

Ideally:

- The video clips contain full phrases along with per-sign timestamp annotations.
- There are more than 30 common phrase sequences.
- There are more than 1 sign per phrase.
- There are more than 10 unique signers (which should not be overlapping with the ones in the training dataset) having balanced ages, genders, hand sizes, etc.
- Each video clip is captured using a front-facing mobile phone camera (selfie mode).
- Each video clip is tightly trimmed to the signs only, with no useless idle time.
- Each video clip has high resolution and reasonable frame rate, yet is not bulky in file size.
- The video clips contain various real-world backgrounds (home, office, school, restaurant, clinic, etc.)
- The video clips use a variety of lighting and camera angles.

An ideal schema would be:

Field	Type	Description	Example
video_id	string	Unique identifier for video clip	“wlasl_restaurant_001”
video_path	string	Path to video clip file	“phrases/wlasl_restaurant_001.mp4”
glosses	list[string]	Ordered list of ASL sign labels	[“water”, “please”]
timestamps	list[object]	Ordered list of temporal boundaries (in seconds) per sign within the video	[{“gloss”: “water”, “start”: 0.2, “end”: 1.1}, {“gloss”: “please”, “start”: 1.2, “end”: 1.9}]
signer_id	string	Unique identifier for the signer	“signer_005”
signer_demographics	object	Characteristics of the signer	{“age_range”: “10-20”, “skin_tone”: “dark”, “used_hand”: “left”, “gender”: “female”}
duration	float	Video clip duration (in seconds)	2.0
fps	int	Video clip frame rate	30
num_frames	int	Total number of frames in video clip	60
resolution	string	Video clip resolution	“1280x720”
env_type	string	Type of environment	“restaurant”
light_type	string	Type of lighting	“daylight”
camera_type	string	Type of camera	“phone front”

4. “Negative” Non-ASL Dataset

We need many video clips of “negative” examples (things that are not ASL signs: idle hand movement, scratching, gesturing) such that our model does not misclassify every random hand movement as an ASL sign.

Ideally:

- The video clips contain different random hand gestures, non-ASL gestures, idle motion, no hands visible, and ASL signs beyond simple everyday vocabulary.
- There are more than 500 video clips.

- There are more than 20 unique signers having balanced ages, genders, hand sizes, etc.

An ideal schema would be:

Field	Type	Description	Example
video_id	string	Unique identifier for video clip	“negative_001”
video_path	string	Path to video clip file	“negatives/negative_001.mp4”
clip_type	string	Category of “negative” example	“random gesture”
duration	float	Video clip duration (in seconds)	2.0
fps	int	Video clip frame rate	30
num_frames	int	Total number of frames in video clip	60

2 Part Two: Reality Check

The following table contains a list of datasets we could use for our project:

Relevant Item	Description	Commentary
ASL Citizen	The first crowdsourced isolated sign language video dataset, containing about 83.4k video recordings of 2.7k isolated signs from American Sign Language (ASL) recorded by 52 Deaf or hard of hearing signers (with consent). Each instance consists of a video file in .mp4 format, an associated gloss (or English transliteration), and an associated anonymous user identifier (1 of 52 signers). Instances are labeled as either “train” (training set), “val” (validation set), or “test” (test set), containing 40,154, 10,304, and 32,941 videos respectively. The data splits are stratified by	This dataset can be used as our project’s primary training set, validation set, and test set, because it already has recommended data splits. We believe this dataset is most relevant to our project since: 1) It has a larger vocabulary size (e.g. 2,731 from ASL Citizen vs 2,000 from WLASL-2000). 2) The data is crowdsourced so it contains examples of everyday signers in everyday environments: videos are self-recorded and come from webcams in uncontrolled settings —

	user such that each user is unseen in the other data splits. The distribution of signers across splits was chosen to balance female-male gender ratio.	varied lighting, cluttered backgrounds, different camera qualities. This is closer to how our app will be used in practice.
WLASL (Word-Level ASL)	The largest scraped video dataset for Word-Level American Sign Language (ASL) recognition, which features 2,000 common different words in ASL. Each video is annotated with a single gloss label and temporal boundaries (start/end frame of the sign). Comes in subsets: WLASL-100, WLASL-300, WLASL-1000, WLASL-2000.	This dataset could be used as a supplementary training set. It covers our target words (for instance, the subset WLASL-100 (containing the 100 most common glosses) overlaps significantly with our scenario vocabulary), and the temporal boundary annotations are critical for our video classifier — they allow us to extract tightly trimmed clips rather than having to process full YouTube videos (much more complicated).
MS-ASL (Microsoft ASL)	The first real-life large-scale sign language data set comprising over 25,000 annotated videos with over 200 signers, covering a large class count of 1000 signs in challenging and unconstrained real-life recording conditions. It has clean labelling and provides temporal annotations. Comes in subsets: MS-ASL100, MS-ASL200, MS-ASL500, MS-ASL1000 (each includes their own train, validation, and test sets).	This dataset could be used as a supplementary training set. It covers our target words (for instance, the subset MS-ASL100 (containing the 100 most common glosses) overlaps significantly with our scenario vocabulary), the sign videos are captured in a range of lightings and backgrounds (realistic settings), and its 222 distinct signers make it the most signer-diverse dataset available (this is important since our model must generalize to new users it has never seen).

Compared to our list of aspirational datasets, we believe that the above three publicly available datasets contain most of everything we need. There is in fact a large number of video clips that are sign-isolated, shot in realistic settings (various lightings, backgrounds, camera types, etc.), recorded by diverse signers, and annotated with temporal boundaries that we can use for our project.

Unfortunately, based on our research, there aren't any "scenario-specific phrase sequence" ASL datasets or any "negative" non-ASL datasets that are publicly available for us to use.

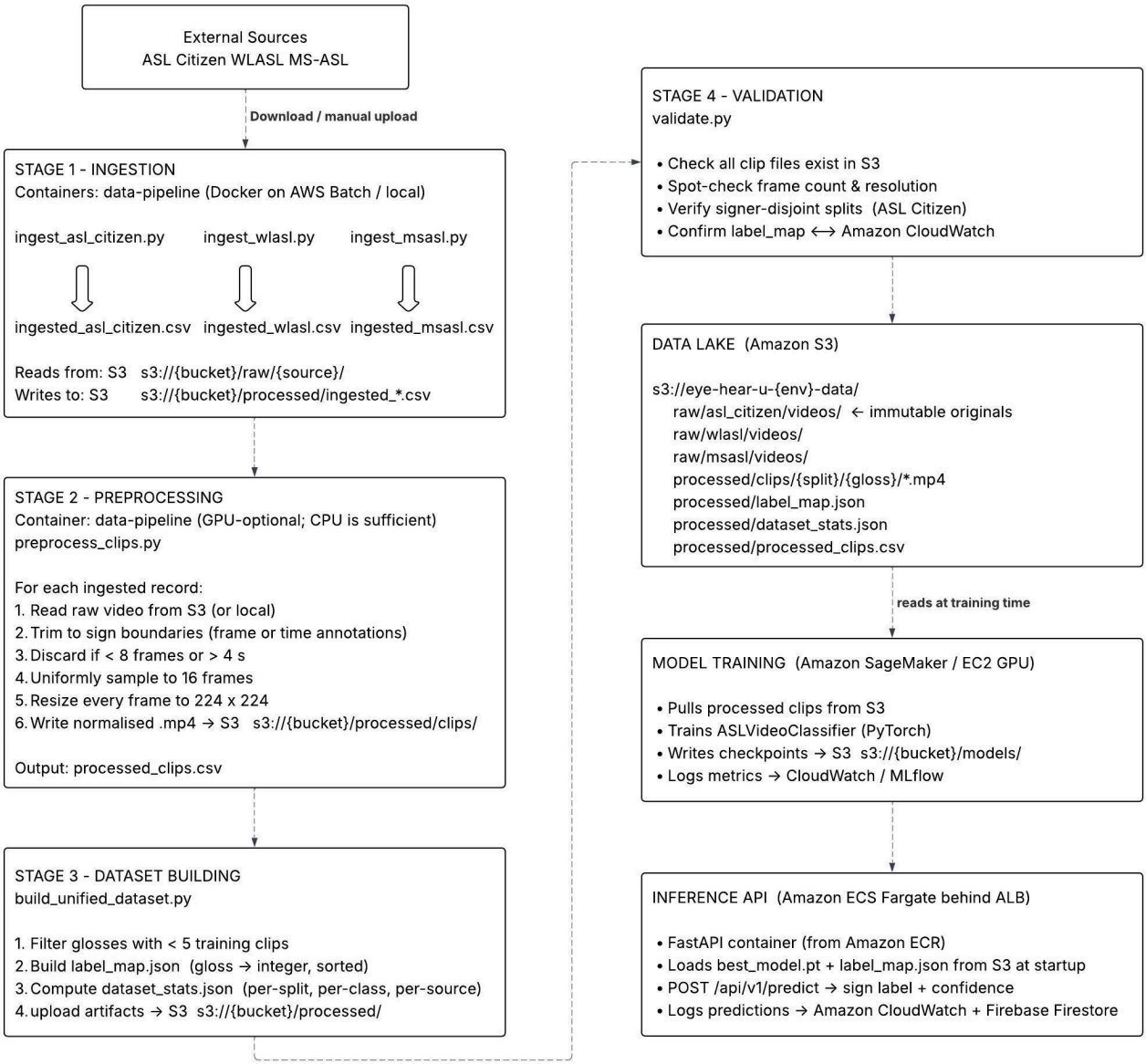
3 Part Three: Data-processing pipelines

Our data pipeline transforms three public ASL video datasets (ASL Citizen, WLASL, MS-ASL) into a single unified, preprocessed dataset that our video classifier (3D CNN pretrained on Kinetics-400) consumes during training and evaluation. The pipeline runs locally during development and on AWS in staging/production, orchestrated by Apache Airflow and deployed via Terraform.

Data Schema

```
┌──────────────────────┐          ┌──────────────────────────┐
│   SignVideo          │          │   SignGloss              │
│──────────────────────│          │──────────────────────────│
│ video_id    PK       │          │ gloss_id    PK           │
│ gloss_id    FK ──────┼──────────▶│ gloss_name               │
│ dataset_source       │          │ category                 │
│ signer_id   FK       │          │ difficulty               │
│ duration_sec         │          │ reference_video_url      │
│ resolution           │          │ num_samples              │
│ fps                  │          │ created      timestamp   │
│ file_path            │          └──────────────────────────┘
│ file_size_bytes      │                     │
│ num_frames    int    │                     │
│ split         enum   │  ← train/val/test   │
│ is_valid    boolean  │                     │
│ ingested_at timestamp│          ┌──────────▼───────────────┐
└──────────────────────┘          │   LabelMap               │
          │                       │──────────────────────────│
          │                       │ gloss_id    PK FK        │
          │                       │ label_index  int         │
          │                       │ version      varchar     │
          │                       │ created_at   timestamp   │
          ▼                       └──────────────────────────┘
┌──────────────────────┐
│   ProcessedClip      │
│──────────────────────│          ┌──────────────────────────┐
│ clip_id     PK       │          │   Signer                 │
│ video_id    FK       │          │──────────────────────────│
│ gloss_id    FK       │          │ signer_id    PK          │
│ num_frames    int    │          │ dataset_source           │
│ height        int    │          │ is_native_signer bool    │
│ width         int    │          │ consent_status           │
│ clip_path     varchar│          └──────────────────────────┘
│ file_size_bytes int  │
│ processed_at timestamp│
└──────────────────────┘
```

Pipeline diagrams with the technologies you are using:



Technologies Used:

- Pipeline diagram — technologies per stage

Stage	Technologies	Inputs	Outputs

1. Ingestion	Python, Requests, boto3, zipfile (S3 range requests)	Raw archives / metadata URLs, S3 **raw/**	**ingested_*.csv** in S3 **processed/** or **processed/mvp/**
2. Preprocessing	Python, OpenCV (cv2), NumPy, boto3	Ingested CSV, raw videos (local or S3)	**processed_clips.csv**, **clips/{train,val,test}/{gloss}/*.mp4**
3. Dataset building	Python, JSON, boto3	**processed_clips.csv**	**label_map.json, dataset_stats.json**
4. Validation	Python, OpenCV, boto3, CloudWatch	**processed_clips.csv, label_map.json**, clip files	Pass/fail report; CloudWatch metric
Data lake	Amazon S3	—-	**raw/, processed/, processed/mvp/, models/**
Training	PyTorch, torchvision (3D CNN), Python	Processed clips + label map	**best_model.pt, label_map.json** in **ml/checkpoints/** or S3
Inference	FastAPI, PyTorch, OpenCV, boto3	Video upload, model + label map	**POST /api/v1/predict** → sign label + confidence

- Pipeline & Data Processing

Component	Technology	Why
Video I/O	OpenCV (cv2)	Cross-platform, pip-installable, handles mp4 read/write
Array manipulation	NumPy	Standard for frame arrays and uniform sampling

HTTP downloads	Requests	Lightweight, for fetching metadata JSONs from GitHub
Data formats	CSV + JSON	Human-readable, versionable, no database needed
Pipeline scripting	Python 3.11+	Same language as the ML stack; shared with PyTorch
ML data loading	PyTorch Dataset/DataLoader	Direct integration with the training loop
Containerisation	Docker	Reproducible environments; required for AWS Batch and ECS
Pipeline orchestration	Apache Airflow	DAG-based scheduling; task dependencies; retries; logs; open source
Dataset versioning	DVC (Data Version Control)	Tracks large data files alongside Git; supports S3 remotes
Experiment tracking	MLflow	Logs training metrics, hyperparameters, and model artifacts

- AWS Cloud Services

AWS Service	Role in Pipeline
Amazon S3	Data lake for raw videos, processed clips, metadata CSVs, label maps, model checkpoints. Two-layer structure: **raw/** (immutable) and **processed/** (derived). S3 versioning enabled for disaster recovery.
Amazon ECR	Docker container registry. Stores the **data-pipeline** image (for ingestion/preprocessing) and the **backend-api** image (FastAPI inference server).
AWS Batch	Runs pipeline stages (ingestion, preprocessing, building, validation) as containerised batch jobs. Jobs pull images from ECR and read/write to S3. Automatically provisions and scales compute.
Amazon SageMaker	Model training on GPU instances (e.g., **ml.g4dn.xlarge)**. Reads processed clips from S3, trains the PyTorch video classifier, writes checkpoints back to S3.

Amazon ECS (Fargate)	Hosts the FastAPI inference API as a serverless container behind an Application Load Balancer. Loads the trained model from S3 at startup.
Amazon CloudWatch	Centralised logging for all pipeline jobs, training runs, and API requests. Custom metrics for pipeline pass/fail status, validation results, and inference latency. Alarms trigger SNS notifications on failure.
Amazon SNS	Sends email/Slack notifications on pipeline completion, validation failures, or training completion.
AWS IAM	Fine-grained access control. Separate roles for: pipeline jobs (S3 read/write), training jobs (S3 + SageMaker), inference API (S3 read-only + CloudWatch write), and CI/CD.
Amazon DynamoDB	Stores pipeline run metadata (run ID, status, timestamps, clip counts) for auditing and debugging. Lightweight alternative to a full relational DB.

When the Pipelines Run and for Which Use Cases

When	Use case
One-time (dataset preparation)	Produce the full or MVP processed dataset for model development.
After adding a source	Re-ingest and preprocess when new raw videos (e.g. WLASL, MS-ASL) are uploaded.
MVP-only (recommended first)	Ingest and preprocess only the MVP vocabulary; outputs go to **processed/mvp/** (smaller, faster).
Post–data change	Re-run build and validate after preprocessing to refresh label map and stats and confirm integrity.

Scheduled / on-demand	Validation can run on a schedule (e.g. weekly) or after training; jobs can be submitted via sbatch/Batch.

Code for an initial version of this pipeline:

All source code lives in data/scripts/:

File	Stage	Description
pipeline_config.py	Shared config	Paths (local + S3), URLs, preprocessing constants, environment
ingest_asl_citizen.py	Ingestion	Parse ASL Citizen metadata, validate, write normalised CSV
ingest_wlasl.py	Ingestion	Download WLASL JSON, parse instances, validate, write CSV
ingest_msasl.py	Ingestion	Download MS-ASL JSONs, parse splits, validate, write CSV
preprocess_clips.py	Preprocessing	Trim → sample 16 frames → resize 224×224 → write .mp4
build_unified_dataset.py	Building	Filter rare glosses, create label_map.json, dataset_stats.json
validate.py	Validation	File existence, frame/resolution checks, signer-leak detection
filter_to_mvp.py	Ingestion	Filter ingested ASL Citizen CSV to MVP vocabulary only
extract_mvp_videos_from_zip.py	Ingestion	Extract only MVP clips from archive.zip in S3 to raw/asl_citizen/videos/
mvp_glosses.txt	Config	One gloss per line for MVP (greetings, needs, restaurant, medical, A–Z, 1–10)
requirements.txt	Dependencies	opencv-python, numpy, requests, boto3

Supporting Files:

File	Description
data/Dockerfile	Container image for all pipeline stages (Python 3.11 + OpenCV + boto3)
ml/training/dataset.py	ASLVideoDataset — reads processed clips, returns (C,T,H,W) tensors
ml/config.py	Dataclass config for model, data, and training hyperparameters

Next steps for features you did not implement in the writeup:

- Automated WLASL & MS-ASL Video Downloading

Currently, ingestion scripts assume that raw video files have already been uploaded to S3 (or placed locally). A future enhancement is to automate video downloading:

- **WLASL / MS-ASL:** Videos originate from YouTube. We plan to integrate `yt-dlp` to download clips by URL and trim to the annotated temporal boundaries automatically.
- **Handling dead links:** Both WLASL and MS-ASL contain a significant proportion of broken YouTube URLs. The pipeline should log failures and report a coverage summary.

- Gloss Vocabulary Alignment Across Datasets

Different datasets may use slightly different gloss labels for the same sign (e.g., "bathroom" vs. "restroom"). A future step is to build a synonym mapping table so that supplementary clips from WLASL and MS-ASL are correctly matched to the ASL Citizen gloss vocabulary.

- DVC Integration for Dataset Versioning

DVC would track large data files alongside Git, with S3 as the remote backend:

dvc init

dvc remote add -d s3remote s3://eye-hear-u-prod-data/dvc/

dvc add data/processed/

git add data/processed/.dvc

This gives the team reproducible dataset versions tied to Git commits.

- Negative Example Pipeline

As described in Part One, a negative example dataset is important for reducing false positives:

- Ingest non-ASL gesture videos (from datasets like Jester or Cambridge Hand Gesture).
- Add a special "__negative__" class to the label map.
- Include negative clips in the training set.

- CI/CD Validation

GitHub Actions workflow that:

- Runs `validate.py` on every pull request to `main`.
- Builds and pushes the `data-pipeline` Docker image to ECR on merge.
- Runs `terraform plan` on infrastructure changes (no auto-apply).

- Frame-Level Feature Caching

Cache per-frame CNN features (from a frozen backbone) as `.npy` arrays in S3 to eliminate redundant forward passes during fine-tuning, reducing epoch time by approximately 3–5×.

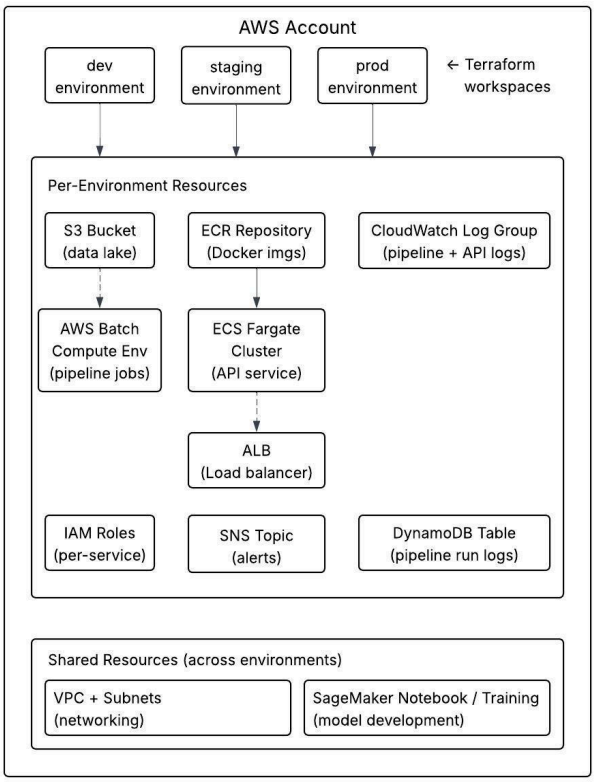
4 Part Four: Infrastructure as Code Implementation

- Infrastructure as code

Tool	Role
Terraform	Provisions all AWS resources (S3 buckets, ECR repos, Batch compute environments, ECS clusters, IAM roles, CloudWatch log groups, SNS topics).

	Supports multiple environments (**dev, staging, prod**) via Terraform workspaces or variable files.
Docker Compose	Local development: runs the full pipeline + backend + Airflow on a single machine.

Infrastructure Architecture:



5 Part 5 Disaster Recovery Demonstration

To simulate a full production failure, we executed terraform destruction on our development environment. This removed all provisioned infrastructure, including the AWS Batch compute environment, ECS cluster, IAM roles, S3 data buckets, and supporting services. During

teardown, AWS Batch failed to delete its ECS cluster due to a missing ecs:DeleteCluster permission in the Batch execution role, causing the compute environment to enter an INVALID state. This reflects a realistic infrastructure failure scenario involving insufficient IAM permissions.

We resolved the issue by updating our Infrastructure as Code to explicitly grant the required ECS deletion permissions to the Batch execution role. After applying the updated Terraform configuration (terraform apply), we re-ran terraform destroy, which successfully removed all resources. This demonstrated controlled remediation entirely through IaC, without manual AWS console intervention.